

Programação de Sistemas Linux

Aula 04 — Sistema de Arquivos Linux

Hugo Marcondes

Departamento Acadêmico de Eletrônica
DAELN

hugo.marcondes@ifsc.edu.br



Objetivos da aula

Ao final desta aula, você será capaz de:

- ✓ Explicar a hierarquia **FHS** e onde cada tipo de arquivo do sistema mora
- ✓ Distinguir os **tipos de arquivo** do Linux (regular, diretório, link, dispositivo, pipe, socket)
- ✓ Interpretar e alterar **permissões**, incluindo SUID, SGID e *sticky bit*
- ✓ Usar as APIs **POSIX** `open`, `read`, `write`, `close` e `stat` em C

Regra de hoje

Tudo no Linux é (ou parece ser) um arquivo. Entender o sistema de arquivos é entender metade do sistema operacional.

Notes

Notes



FHS: a hierarquia de diretórios

Notes

Por que um padrão de diretórios?

Definição

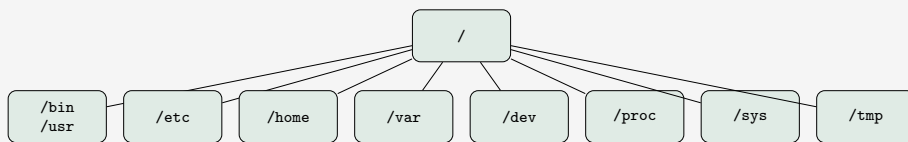
FHS (*Filesystem Hierarchy Standard*) é o padrão que define **onde** cada tipo de arquivo deve morar em um sistema Linux — binários, configurações, bibliotecas, dados de usuário, arquivos temporários.

- Sem ele, cada distribuição colocaria as coisas em lugares diferentes
- Com ele, você sabe que `/etc` tem configuração em **qualquer** distribuição Linux — Ubuntu, Fedora, Arch...
- Scripts, pacotes e ferramentas de automação **dependem** dessa previsibilidade

Notes



A raiz e seus principais galhos



- /proc e /sys não guardam nada em disco – são **janelas virtuais** para o kernel (vimos isso na Aula 2)
- /home é o único galho que é “seu” – o resto é do sistema



Notes

Os diretórios que você mais vai usar

Diretório	O que tem lá
/bin, /usr/bin	executáveis do sistema e dos programas instalados
/etc	arquivos de configuração (texto puro, editáveis)
/home/usuario	arquivos pessoais de cada usuário
/var	dados que mudam com o tempo: logs, cache, filas
/tmp	arquivos temporários – pode ser limpo a qualquer reboot
/dev	arquivos especiais que representam dispositivos
/proc, /sys	sistemas de arquivo virtuais, gerados pelo kernel



Notes

```
1 $ cat /proc/cpuinfo | head -5      # informacoes da CPU, geradas na hora
2 $ cat /proc/1/comm                 # nome do processo com PID 1
3 $ ls /sys/class/thermal/           # sensores de temperatura, se
    existem
4
```

Por que cat funciona em algo que não existe?

Quando você lê esses arquivos, o **kernel** intercepta a leitura e gera o conteúdo **na hora**, a partir do seu estado interno. Não há bytes gravados em disco — é a mesma ideia do `strace`, só que para inspecionar o kernel em vez de um processo.



Onde está montado o quê

```
1 $ df -h                          # espaco livre/usado por sistema de arquivos
    montado
2 $ mount | grep ^/dev              # o que esta montado e onde
3 $ findmnt /                        # mostra o sistema de arquivos raiz
```

- Um **sistema de arquivos** (ext4, btrfs, tmpfs...) é **montado** em um ponto da árvore — é isso que une discos/partições físicas à hierarquia FHS que você vê
- /proc e /sys também aparecem no `mount` — são sistemas de arquivo, só que virtuais (proc, sysfs)



Tipos de arquivo no Linux

Notes

Sete tipos, um só conceito

Indicador	Tipo	Exemplo
-	regular	um .c, um .txt, um binário
d	diretório	/home, /etc
l	link simbólico	atalho para outro caminho
b	dispositivo de bloco	/dev/sda (disco)
c	dispositivo de caractere	/dev/tty, /dev/urandom
p	pipe nomeado (FIFO)	criado com mkfifo
s	socket	usado em IPC/rede

No Linux, “tudo é um arquivo” — inclusive um disco ou um socket de rede.

Notes



ls -l: o primeiro caractere não é acaso

```
1 $ ls -l /etc/passwd /home /dev/sda /dev/tty
2 -rw-r--r--  1 root root  2210 ago 10 09:00 /etc/passwd
3 drwxr-xr-x  3 root root  4096 ago 10 09:00 /home
4 brw-rw----  1 root disk   8, 0 ago 10 09:00 /dev/sda
5 crw-rw-rw-  1 root tty    5, 0 ago 10 09:00 /dev/tty
```

- O **primeiro caractere** da coluna de permissões é o tipo do arquivo
- Em dispositivos, os números 8, 0 são o **major/minor** — identificam *qual* driver e *qual* dispositivo (assunto de aulas futuras)



Notes

Link simbólico vs. *hard link*

```
1 $ ln -s original.txt atalho.txt # link simbolico: aponta para um
   CAMINHO
2 $ ln original.txt copia.txt      # hard link: aponta para o mesmo INODE
3 $ ls -li original.txt copia.txt atalho.txt
```

- **Link simbólico:** um arquivo à parte que guarda um caminho — se o original sumir, o link fica **quebrado**
- **Hard link:** outro nome para o **mesmo inode** — os dois “arquivos” são, na prática, o mesmo conteúdo
- Hard link **não funciona entre sistemas de arquivo diferentes**; link simbólico funciona sempre



Notes

Permissões: rwx, SUID, SGID, sticky bit

Notes

rwx: três permissões, três donos

Notes

	dono (u)	grupo (g)	outros (o)
ler (r)	4	4	4
escrever (w)	2	2	2
executar (x)	1	1	1

- `chmod 755 script.sh` $\equiv$ `rxr-xr-x` (soma: 4+2+1, 4+1, 4+1)
- `chmod 644 dados.txt` $\equiv$ `rw-r-r-` — típico para arquivos de dados
- Em um **diretório**, x significa “pode entrar/listar”, não “executar”



```
1 $ chmod 750 script.sh      # numerico: rwxr-x---
2 $ chmod u+x script.sh      # simbolico: adiciona execucao para o dono
3 $ chmod go-w dados.txt     # remove escrita de grupo e outros
4 $ chown aluno:turma dados.txt # troca dono e grupo
5
```

- Forma **numérica** (750): rápida, mas você precisa calcular a soma
- Forma **simbólica** (u+x): você só muda o que precisa, sem afetar o resto
- **chown** normalmente exige privilégios de **root** (ou ser o dono atual)



Além do `rwx`: SUID, SGID e *sticky bit*

SUID (*Set User ID*)

Ao executar, o processo roda com os privilégios do **dono do arquivo**, não de quem o executou. É como o `passwd` consegue alterar `/etc/shadow` mesmo rodando como usuário comum.

SGID e *sticky bit*

SGID em diretório: novos arquivos criados ali herdam o *grupo* do diretório. **Sticky bit** em diretório (como `/tmp`): só o **dono do arquivo** pode apagá-lo, mesmo que outros tenham permissão de escrita na pasta.



```
1 $ ls -l /usr/bin/passwd
2 -rwsr-xr-x 1 root root 64152 ... /usr/bin/passwd
3           ^ 's' no lugar do 'x' do dono = SUID ativo
4
5 $ ls -ld /tmp
6 drwxrwxrwt 15 root root 4096 ... /tmp
7           ^ 't' no lugar do 'x' de outros = sticky bit
```

- `chmod u+s` programa ativa SUID; `chmod +t` pasta ativa o *sticky bit*
- SUID em binários **gravável por qualquer um** é um risco sério de segurança — por isso é raro e sempre revisado com cuidado



APIs POSIX de arquivo

Notes

Notes

Por trás do fopen: as syscalls de arquivo

- `fopen`, `fprintf`, `fclose` são funções da **libc** — convenientes, com buffer, portáteis
- Por baixo, elas chamam as **syscalls POSIX**: `open`, `read`, `write`, `close`
- Hoje vamos usar as syscalls **diretamente** — útil quando você precisa de controle fino (permissões na criação, sem buffer, etc.)

Prévia da próxima aula

A relação exata entre função de biblioteca e chamada de sistema — e o que acontece quando uma syscall falha — é o assunto da Aula 5.



Notes

open, read, write, close

```
1 int fd = open("dados.txt", O_RDONLY);
2 if (fd == -1) {
3     perror("open");
4     return 1;
5 }
6
7 char buffer[128];
8 ssize_t n = read(fd, buffer, sizeof(buffer));
9
10 close(fd);
```

- `open()` retorna um `int` — o **descritor de arquivo** (*file descriptor*)
- Flags combináveis: `O_RDONLY`, `O_WRONLY`, `O_CREAT`, `O_TRUNC`
- `O_CREAT` exige um terceiro argumento: o `mode_t` (permissões do novo arquivo)



Notes

stat(): metadados sem abrir o arquivo

```
1 struct stat info;
2 if (stat("dados.txt", &info) == -1) {
3     perror("stat");
4     return 1;
5 }
6
7 printf("Tamanho: %ld bytes\n", (long) info.st_size);
8 printf("Inode:   %lu\n", (unsigned long) info.st_ino);
9 printf("Modo:    %o\n", info.st_mode & 0777);
```

- struct stat traz tamanho, inode, permissões, dono, timestamps...
- lstat() é igual, mas **não segue** links simbólicos — útil quando você quer informação sobre o link em si



Notes

opendir/readdir: percorrendo um diretório

```
1 DIR *dir = opendir(".");
2 struct dirent *entrada;
3
4 while ((entrada = readdir(dir)) != NULL) {
5     struct stat info;
6     lstat(entrada->d_name, &info);
7     printf("%s (%ld bytes)\n", entrada->d_name,
8           (long) info.st_size);
9 }
10
11 closedir(dir);
```

- readdir() devolve **um** dirent por chamada — inclusive . e ..
- Combine com stat/lstat para saber **tipo, tamanho e permissões** de cada entrada — exatamente o que vocês vão implementar no laboratório



Notes

Encerramento

Notes

Síntese da aula

Notes

Conceito	Pergunta que responde
FHS	Onde esse tipo de arquivo deveria estar?
Tipos de arquivo	O que é essa entrada: arquivo, diretório, link, dispositivo?
Permissões / SUID/SGID/sticky	Quem pode ler, escrever, executar ou apagar isso?
APIs POSIX (<code>open/stat/...</code>)	Como meu programa lê esses metadados?

“Tudo é um arquivo” só faz sentido com FHS, tipos, permissões e POSIX juntos.



Atividade prática de hoje

No laboratório, vocês vão:

- 1 Escrever um programa em C que lista um diretório com `opendir/readdir`
- 2 Usar `stat/lstat` para mostrar tipo, permissões, tamanho e `inode` de cada arquivo
- 3 Explorar `chmod`, SUID, SGID e *sticky bit* na prática

Guia da atividade

Todas as instruções, código-fonte e critérios de avaliação estão no **guia de atividade prática** do Laboratório 03.



Notes

Próxima aula

Aula 5 – Chamadas de Sistema: Fundamentos

- Mecanismo de *syscall*: como o processo entra no kernel
- *Syscall* vs. função de biblioteca (e por que `printf` não é uma)
- `errno` e tratamento de erros
- A `libc` como camada de conveniência sobre as *syscalls*

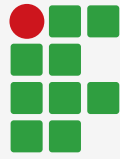
Bibliografia de apoio

- Manuais: `man 7 hier`, `man 2 stat`, `man 2 open`, `man 7 inode`
- The Linux Documentation Project – Filesystem Hierarchy Standard
- HALLINAN, C. *Embedded Linux Primer* – capítulo sobre sistema de arquivos



Notes

That's all folks!



**INSTITUTO
FEDERAL**
Santa Catarina

Câmpus
Florianópolis



Notes

Notes
